



TraceBack

Lost & Found Item Tracker

SIH Presentation & Technical Q&A Guide

A centralised, AI-assisted platform where people report lost or found items, receive smart match suggestions, prove ownership privately, chat securely and track an item all the way back to its owner. This guide covers the architecture, workflow, technology choices and the questions a jury is most likely to ask — with answers.

Node.js + Express

React + Vite

SQLite

Weighted matching engine

Perceptual image hashing

JWT auth

1 Problem & objective

On any campus, in any office or public place, lost items and found items are reported through scattered WhatsApp groups, notice boards and lost-and-found desks that never talk to each other. An owner and a finder can be metres apart and never connect.

TraceBack gives both sides one place to report, automatically matches lost against found, verifies the real owner, and tracks the item to a confirmed return.

Who it's for

Students, staff and visitors on a campus; adaptable to offices, transit hubs and events.

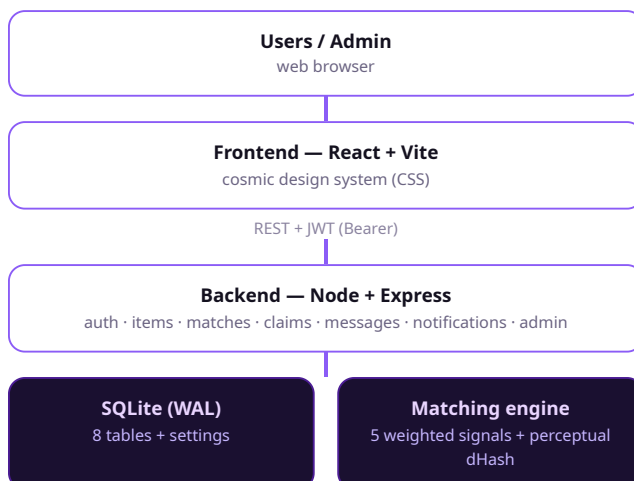
Core promise

Report once. The engine keeps scanning every new report and alerts you the moment a likely match appears.

What makes it safe

Private ownership questions and masked identities stop a stranger from simply claiming your things.

2 System architecture



Request lifecycle

1. The React client calls the REST API with a JWT in the [Authorization](#) header.
2. Express validates the token, runs the route, and reads/writes SQLite through prepared statements.
3. On a new/edited report the **matching engine** runs synchronously, persists any matches and fires notifications.
4. The client polls notifications and messages so alerts appear without a page refresh.

Why one Node service (not micro-services)

A single process is far easier to run, demo and reason about under hackathon time pressure. The code is already split into clean modules (auth, items, matches, claims, ...), so any one could be extracted into its own service later without a rewrite.

The matching engine is a pure, dependency-free module — it can be lifted into a separate Python/FastAPI service if heavier ML is ever needed, without touching the API contract.

3 Technology stack & why

LAYER	CHOICE	WHY THIS, FOR THIS PROJECT
Frontend	React 18 + Vite + React Router	Component reuse across 23 screens; Vite gives instant HMR and a small production build.
Styling	Hand-built CSS design system (tokens › components)	Full control of the cosmic glow theme with zero framework weight; one token file re-themes everything.
Backend	Node.js + Express (ES modules)	The recommended stack for this problem statement; huge ecosystem, tiny boilerplate.
Database	SQLite — built-in <code>node:sqlite</code> , or <code>better-sqlite3</code>	Same relational model as PostgreSQL but zero setup . All queries are plain SQL, so moving to Postgres is a driver swap, not a rewrite.
Auth	JWT + bcrypt password hashing	Stateless, role-aware (user / admin); no session store to run.
Images	<code>multer</code> upload + <code>sharp</code> 8×8 dHash	Real perceptual image similarity computed locally — no cloud key, works offline.
Charts	Hand-rolled SVG	On-theme gradient fills, no chart dependency to ship.

Robustness detail worth mentioning to judges: the database auto-selects a driver — Node's built-in `node:sqlite` on Node 22.5+ (no compiler needed) and `better-sqlite3` when present. Both native modules are *optional*, so `npm install` can never fail for lack of a C++ toolchain.

4 Complete user workflow

The end-to-end journey, exactly as demonstrated:



Item status lifecycle (enforced server-side, never moves backward)



Lost item example

A student loses a black wallet in the canteen › clicks **Report Lost**, enters name, category, description, location, date/time and a photo › the engine searches all open *found* reports › a high-scoring match surfaces and both people are notified.

Found item example

Another student finds the wallet › clicks **Report Found**, adds details plus **private ownership questions** only the true owner could answer › the engine compares it against open *lost* reports › possible owners are alerted.

5 The smart matching engine

It never relies on an exact item name. It scores five independent signals, each 0–100%, and combines them with configurable weights:

Item / category similarity		25%
Description similarity		25%
Location similarity		20%
Date / time proximity		15%
Image similarity		15%

The techniques, in one line each

- **Synonym lexicon** — collapses `phone`↔`mobile`↔`cell` , `wallet`↔`purse` , `canteen`↔`cafeteria`↔`mess` , colour families and place names.
- **Text similarity** — token overlap + Jaccard + character-bigram cosine, so typos and different wording still match.
- **Calibration curve** — two people describing the same object share only 30–45% of words; a curve maps that onto human-meaningful confidence.
- **Date proximity** — exponential decay across a configurable window (default 14 days).
- **Image similarity** — 64-bit perceptual dHash; similarity = Hamming distance between the two hashes.
- **Weight re-normalisation** — a factor with no data (e.g. no photo) is dropped and the remaining weights rescale, so scores stay comparable.

Worked example (from the live demo)

Black wallet, same canteen, same afternoon, similar description ›

Category & name	81%
Description	75%
Location	100%
Date / time	100%
Image	94%
Combined	88% — Possible Match

Weights are live-tunable from Admin › Settings and persisted, so the engine is genuinely configuration-driven.

6 Ownership verification & security

A major safeguard prevents anyone from simply claiming an item:

1. The finder stores **private question/answer pairs** when reporting.
2. The API returns **only the questions** — answers never leave the database.
3. A claimant's answers are auto-scored against the stored answers by the same text-similarity engine.
4. The score is **advisory**; the finder or an admin makes the final call.
5. A rejected claimant can raise a **dispute** that lands in the admin queue.

In the demo the genuine owner scores **~89%** while an impostor scores **~2%**.

Privacy & security controls

- **JWT** auth; passwords hashed with **bcrypt**.
- Reporter names masked (**Aa...** S.) and emails masked in every public response.
- Phone numbers never serialised through the API.
- Identities unlock **only after a claim is approved**.
- Role-based access — admin routes reject non-admins (403); protected routes reject anonymous requests (401).
- You cannot claim your own report; server-side input validation on every write.

7 Database design

TABLE	KEY FIELDS
users	id, name, email, password_hash, role, phone, campus, avatar_hue, status, created_at
items	id, user_id, type(lost/found), title, category, description, location, item_date, image_url, image_hash, status, questions, is_flagged, created_at, updated_at
matches	id, lost_item_id, found_item_id, match_score, breakdown, status (unique per pair)
claims	id, item_id, match_id, claimant_id, proof, answer_score, stage, status, decided_by, decision_note
messages	id, sender_id, receiver_id, item_id, message, read_status, created_at
notifications	id, user_id, type, title, message, link, read_status, created_at
disputes	id, claim_id, raised_by, reason, status, resolution
settings	key, value — live matching-engine configuration

Foreign keys enforced; indexes on item type/status, per-user lookups and match sides. WAL journaling for concurrent reads.

8 REST API (surface)

ENDPOINT	PURPOSE	ENDPOINT	PURPOSE
POST /auth/register · /login	Account + JWT	POST /claims	Open a claim
GET/PATCH /auth/me	Profile	POST /claims/:id/verify	Answer + auto-score
GET /items	Browse + filters	POST /claims/:id/decision	Approve / reject
POST /items	Create report (multipart)	POST /claims/:id/handover	Mark RETURNED
GET /items/:id	Item + matches + questions	GET/POST /messages/*	Item-scoped chat
POST /items/:id/rescan	Re-run engine	GET /notifications	Alert centre
GET /matches	Your match pairs	GET/PUT /admin/*	Moderation + analytics + settings

9 MVP vs advanced features (all built)

MVP

- Login / register with roles
- Report lost / found with photo
- Search & filter
- Weighted match score
- Claim verification
- Item status tracking
- Admin dashboard

Advanced differentiators

- Perceptual image similarity (dHash)
- Synonym-aware, calibrated text matching
- In-app chat with item context
- Live notification centre
- Auto-scored verification + disputes
- Analytics (trends, donut, hotspots, funnel)
- Live engine weight tuning

10 Anticipated jury questions & answers

Concept & impact

Q How is this better than a WhatsApp lost-and-found group?

A A group is an unsearchable, unstructured feed with no matching, no verification and no status tracking. TraceBack structures every report, actively matches lost against found, proves ownership before handover, and shows exactly where each case stands.

Q Who is the target user and how do you measure success?

A Students, staff and visitors on a campus. Success metrics are already in the analytics dashboard: match success rate, claim approval rate and average resolution time.

The matching engine

Q Is this actually "AI"? What algorithm is it?

A It's a transparent, weighted multi-signal scoring model — a lightweight, explainable recommendation engine rather than a black box. Text uses token-overlap, Jaccard and character-bigram cosine with a synonym lexicon and a calibration curve; images use a perceptual difference hash. Every score is fully explainable, which is exactly what a verification workflow needs.

Q Why not a deep-learning / sentence-transformer model?

A It's an intended upgrade path — the engine is an isolated module behind a stable interface. For a reliable, offline, instant demo, a transparent scoring model is more defensible and needs no GPU or API key. We can drop in embeddings later without changing the API.

Q What if there's no photo?

A The image factor is marked unavailable and its 15% weight is redistributed across the remaining four, which re-normalise to 100%. A perfect text+location+time match still reaches a high score.

Q How do you avoid false matches?

A A configurable threshold hides weak pairs, matching is one-to-many for review (not auto-confirmed), and the human verification step with private questions is the real gate before any handover.

Security & trust

Q What stops someone falsely claiming an item?

A Private ownership questions only the true owner can answer (answers never exposed by the API), an advisory auto-score, a mandatory human decision by the finder/admin, masked identities until approval, and a dispute path for contested rejections.

Q How is user data protected?

A Passwords are bcrypt-hashed, sessions are stateless JWTs, names/emails are masked in public responses, phone numbers are never sent, and identities are revealed only after a claim is approved.

Technical & scale

Q Why SQLite — will it scale?

A SQLite handles thousands of concurrent readers comfortably, which covers a campus. Every query is plain SQL, so migrating to PostgreSQL for a multi-campus deployment is a driver change, not a rewrite. It was chosen for zero-setup reliability during the demo.

Q How would you scale to many campuses?

A Swap SQLite for PostgreSQL, put the stateless API behind a load balancer, move uploads to object storage (S3/Cloudinary), and extract the matching engine into its own service if load demands it. Nothing in the design blocks this.

Q Real-time updates?

A Today the client polls notifications and messages on a short interval — simple and demo-safe. WebSockets are the documented next step for instant push.

Q Is it tested?

A Yes — a 44-assertion end-to-end script drives the full journey (register › report both sides › match › genuine claim vs impostor › approve › handover › returned › admin › access control), plus an automated screenshot pass across three screen sizes.

Product & future

Q What would you add next?

A College email verification, map-based location matching (haversine on coordinates), WebSocket push, sentence-transformer text embeddings, and a native mobile app.

11 Live demo script (for judges)

2 min

REPORT LOST + FOUND

1 min

MATCH & NOTIFY

2 min

VERIFY & APPROVE

1 min

RETURN + ADMIN

1. Sign in as **Student A** (the one-click chip) › **Report Lost**: a black wallet at the canteen. Show the live-preview card and the match-strength checklist.
2. Sign in as **Student B** › **Report Found**: the same wallet, adding two private ownership questions. On submit, the engine immediately shows an **88% match**.
3. Open **Possible Matches**: expand the pair to reveal the five factor bars and the reasons. Both users already have a notification.
4. As Student A, **Claim** it and answer the verification questions — show the ~89% auto-score.
5. As Student B, open the claim, review the answers, **Approve**. Identities now unlock; coordinate handover in chat.
6. **Confirm handover** › both reports flip to **RETURNED**, then **CLOSED**.
7. Sign in as **Admin**: show KPIs, the analytics charts and the live matching-weight sliders in Settings.

Punchline for the jury: change a weight in Admin › Settings, re-scan a report, and the score visibly changes — proving the engine is genuinely configuration-driven, not hard-coded for the demo.